

Blind Assist Device

Tinkercad Arduino Uno R3 Project (free version) - Version 2 (silent when idle)

An ultrasonic + PIR based assistant that warns a visually impaired person about potholes or drops, obstacles, living things ahead and road crossings. Alerts use a buzzer for what is ahead and a vibration motor to say that the obstacle is likely alive.

Honest limits: An ultrasonic sensor only measures distance. It can detect obstacles (distance gets shorter) and potholes or drops (distance gets longer), but it cannot recognize a crossing by itself. A photoresistor (LDR) is used here only to *simulate* detecting the bright stripes of a crossing. A PIR sensor detects only *moving* warm bodies, so "living" means "likely living", not a guarantee.

1. Components Needed

Qty	Component (Tinkercad name)	Purpose
1	Arduino Uno R3	Main controller, reads sensors and decides the alert
2	Ultrasonic Distance Sensor (HC-SR04)	One faces forward (obstacles), one angled downward (potholes)
1	PIR Sensor	Detects moving warm bodies (likely living things)
1	Piezo Buzzer	Sound alerts for pothole, obstacle and crossing
1	DC Motor (stand-in for a vibration motor)	Vibration alert when the obstacle is likely living
1	NPN Transistor (2N2222 / similar)	Lets a small Arduino pin switch the motor
1	Diode (1N4007)	Protects the transistor from motor voltage spikes
1	Resistor 1 k ohm	Limits current into the transistor base
1	Photoresistor (LDR)	Simulates detecting a crossing (bright stripes)
1	Resistor 10 k ohm	Pull-down resistor for the LDR voltage divider
1	LED (optional)	Visual indicator for pothole alerts
1	Resistor 220 ohm	Limits LED current
1	Breadboard + jumper wires	Connections

2. Pin Configuration

Component	Pin	Connects to
Forward ultrasonic	VCC / GND	5V / GND
	TRIG	D9
	ECHO	D10
Ground ultrasonic (angled down)	VCC / GND	5V / GND
	TRIG	D11
	ECHO	D12
PIR sensor	5V / GND	5V / GND
	OUT	D2

Component	Pin	Connects to
Piezo buzzer	+ (positive)	D6
	- (negative)	GND
Transistor (NPN)	Base	D5 through the 1 k ohm resistor
	Emitter	GND
	Collector	Motor terminal 1
DC motor (vibrator)	Terminal 2	5V
Diode 1N4007	Across the motor	Stripe (cathode) on the 5V side
LED	Anode (+)	D13 through the 220 ohm resistor
	Cathode (-)	GND
Photoresistor (LDR)	One leg	5V
	Other leg	A0
10 k ohm resistor	One leg / other leg	A0 / GND

Mounting idea: point the forward ultrasonic sensor and the PIR straight ahead. Angle the ground sensor downward so it normally reads a fixed distance to the floor (about 100 cm). In a real build, use a small coin vibration motor in place of the DC motor.

3. Working of Each Component

Arduino Uno R3

The brain of the project. It sends trigger pulses to the ultrasonic sensors, reads their echo timing, reads the PIR (digital) and LDR (analog) inputs, and then drives the buzzer, the vibration motor and the LED according to the priority rules in the code.

Ultrasonic sensor (HC-SR04)

It sends out a short burst of ultrasonic sound (above human hearing) when the TRIG pin is pulsed high for 10 microseconds. The sound bounces off an object and returns. The ECHO pin stays high for as long as the sound took to go out and come back. The Arduino measures that time and converts it to distance: $\text{distance (cm)} = \text{time} \times 0.034 / 2$. The forward sensor finds obstacles. The ground sensor, tilted down, gives a steady floor distance. If that reading suddenly becomes much larger, the floor has dropped away, which means a pothole or step down.

PIR sensor

A Passive Infrared sensor senses changes in infrared radiation. Living bodies give off heat, and when a warm body moves across its field of view, the sensor's output pin goes HIGH. It does not send anything out (it is passive). It detects movement of heat, so a perfectly still person may not trigger it, and other moving warm things such as a vehicle engine can.

Piezo buzzer

A piezo element changes shape when voltage is applied. The Arduino's `tone()` function sends it a square wave of a chosen frequency, so it makes a note of that pitch. Different pitches and beep speeds are used to tell alerts apart.

DC motor / vibration motor

A small motor with an off-centre weight shakes when it spins, which the user feels as vibration. It needs more current than an Arduino pin can supply, so it is switched by a transistor. In Tinkercad a plain DC motor stands in for it.

NPN transistor

Works as an electronic switch. A tiny current into the base (from D5 through the 1 k ohm resistor) lets a much larger current flow from collector to emitter, which powers the motor from 5V.

Diode (1N4007)

When a motor is switched off, its coil produces a voltage spike that can damage the transistor. The diode, placed across the motor, gives that spike a safe path to flow.

Photoresistor (LDR)

Its resistance falls as light gets brighter. Together with the 10 k ohm resistor it forms a voltage divider, so pin A0 sees a voltage that rises with brightness. `analogRead()` turns this into a number from 0 (dark) to 1023 (bright). In this project a high reading stands in for the bright stripes of a crossing.

LED and 220 ohm resistor

The LED lights up during a pothole alert as a visual indicator. The resistor limits current so the LED is not damaged.

4. Full Code

```
const int trigF=9, echoF=10, trigG=11, echoG=12;
const int buzzer=6, ldr=A0, pir=2, vib=5;
int groundBase;           // measured automatically at start
int lightThreshold = 700;
bool crossingActive = false;

long readCm(int trig, int echo) {
  digitalWrite(trig, LOW); delayMicroseconds(2);
  digitalWrite(trig, HIGH); delayMicroseconds(10);
  digitalWrite(trig, LOW);
  long d = pulseIn(echo, HIGH, 30000);
  return d ? d * 0.034 / 2 : 400;
}

void setup() {
  Serial.begin(9600);
  pinMode(trigF, OUTPUT); pinMode(echoF, INPUT);
  pinMode(trigG, OUTPUT); pinMode(echoG, INPUT);
  pinMode(buzzer, OUTPUT); pinMode(13, OUTPUT);
  pinMode(pir, INPUT);
  pinMode(vib, OUTPUT);

  delay(500);
  groundBase = readCm(trigG, echoG); // start position = normal floor
  crossingActive = analogRead(ldr) > lightThreshold; // no beep at start
}

void loop() {
  long front = readCm(trigF, echoF);
  long ground = readCm(trigG, echoG);
  int light = analogRead(ldr);
  bool alive = digitalRead(pir) == HIGH;
  bool pothole = ground > groundBase + 20;
  bool bright = light > lightThreshold;

  Serial.print("front="); Serial.print(front);
  Serial.print(" ground="); Serial.print(ground);
  Serial.print(" light="); Serial.println(light);

  digitalWrite(vib, (front < 100 && alive) ? HIGH : LOW);
  digitalWrite(13, pothole ? HIGH : LOW);

  if (pothole) {
    tone(buzzer, 300, 400); delay(500); // pothole / drop
  }
  else if (front < 100) { // obstacle
    tone(buzzer, 1500, 80);
    delay(80 + map(front, 0, 100, 50, 500));
  }
  else if (bright && !crossingActive) { // crossing: only once
    tone(buzzer, 800, 100); delay(120);
    tone(buzzer, 1000, 100); delay(120);
  }
  crossingActive = bright;
  delay(30);
}
```

5. Code Explanation

5.1 Pin setup

```
const int trigF=9, echoF=10, trigG=11, echoG=12;
const int buzzer=6, ldr=A0, pir=2, vib=5;
int groundBase;
int lightThreshold = 700;
bool crossingActive = false;
```

These give names to the pins so the code is easier to read. F means the forward sensor and G means the ground sensor. Each ultrasonic sensor has a TRIG pin (sends the sound pulse) and an ECHO pin (receives it back). **vib** is the pin that drives the vibration motor through the transistor. **groundBase** is the normal distance from the ground sensor to the floor, in cm. It is no longer typed in by hand: it is measured once at startup. Anything much bigger than this means the floor has dropped away. **lightThreshold** is the LDR reading above which the surface counts as bright (a crossing). **crossingActive** remembers whether it was already bright, so the crossing sound plays only once when the light changes from dark to bright.

5.2 The distance function

`long readCm(int trig, int echo)` works for both sensors, because you tell it which pins to use.

- It sets TRIG low for 2 microseconds to clear it.
- It sets TRIG high for 10 microseconds, which makes the sensor send an ultrasonic burst.
- `pulseIn` measures how long ECHO stays high, which is the time for the sound to travel out and bounce back. The 30000 is a timeout in microseconds, so it stops waiting if nothing comes back.
- Distance = time x 0.034 / 2. Sound travels about 0.034 cm per microsecond, and you divide by 2 because the sound goes there and back.
- `d ? ... : 400` means that if d is 0 (no echo), it returns 400 cm, which counts as very far.

5.3 setup()

Runs once. `Serial.begin(9600)` starts the Serial Monitor so you can see live sensor values. TRIG pins send signals, so they are outputs. ECHO pins and the PIR receive signals, so they are inputs. The buzzer, the LED on pin 13 and the vibration pin are outputs. After a short 500 ms wait, the ground sensor reading is stored in **groundBase**, so the position you start the simulation in is treated as the normal floor. The current LDR state is also stored in **crossingActive**, so nothing beeps at start-up.

5.4 loop(): reading the sensors

```
long front = readCm(trigF, echoF);
long ground = readCm(trigG, echoG);
int light = analogRead(ldr);
bool alive = digitalRead(pir) == HIGH;
bool pothole = ground > groundBase + 20;
bool bright = light > lightThreshold;
```

This runs over and over. Each time round it reads the front distance, the ground distance, the light level (0 to 1023) from the LDR, and whether the PIR is triggered (alive). It then works out two yes/no values: **pothole** (the floor is more than 20 cm farther than normal) and **bright** (the light is above the threshold). The `Serial.print` lines show front, ground and light values in the Serial Monitor, which helps you pick the right thresholds.

5.5 loop(): vibration for living things

```
digitalWrite(vib, (front < 100 && alive) ? HIGH : LOW);
digitalWrite(13, pothole ? HIGH : LOW);
```

This line runs at the top of every loop. The motor turns on when an object is within 100 cm **and** the PIR is triggered, and turns off otherwise. Vibration is a separate channel from the buzzer, so the user does not have to remember another sound. The second line lights the LED only while a pothole is detected.

5.6 loop(): the decisions

The if / else if chain is ordered by priority. Only the first true condition runs, so danger comes first.

- **Pothole** (pothole is true): the floor is more than 20 cm farther than normal, so there is a hole or drop. The buzzer plays a low 300 Hz tone for 400 ms, then the loop waits 500 ms.
- **Obstacle** (front < 100): something is within 100 cm. The buzzer plays a short 1500 Hz beep (80 ms), then the loop pauses. The pause is set by map(), so closer objects give faster beeps. If the vibration motor is also running, the obstacle is likely living.
- **Crossing** (bright and not already active): a bright LDR reading, like white stripes on the road, plays two rising tones, 800 Hz then 1000 Hz, **once**. It plays again only after the light has gone dark and then bright again.
- **Otherwise**: nothing is played, so the buzzer stays silent.

5.7 tone() with a duration, and the end of the loop

tone(pin, frequency, duration) plays a sound for a set time (in ms) and then stops by itself. That is why the buzzer no longer needs noTone() and stays silent whenever no condition is true. At the end of every loop, crossingActive = bright; stores the current light state for the next round, and delay(30) gives a small rest between readings.

5.8 The map() function

```
delay(map(front, 0, 100, 50, 500));
```

map(value, fromLow, fromHigh, toLow, toHigh) converts a number from one range into the matching number in another range. Here it turns distance (0 to 100 cm) into a pause between beeps (50 to 500 ms). At 0 cm the pause is 50 ms (rapid beeping), at 100 cm it is 500 ms (slow beeping), and at 50 cm it is 275 ms. The formula is: (value - fromLow) x (toHigh - toLow) / (fromHigh - fromLow) + toLow. map() does not limit results to the output range, but here the condition front < 100 already keeps the input safe.

5.9 Why the buzzer used to sound all the time (fixed in this version)

- The photoresistor starts at a bright default in Tinkercad, so light > 700 was always true and the crossing melody repeated every 200 ms. Now it plays only once when the light changes from dark to bright.
- If the ground sensor started farther than groundBase + 20, the pothole tone played non-stop. Now groundBase is measured at start-up, so the starting position counts as normal floor. Start the simulation with the ground sensor at its normal floor distance.
- Sounds now use tone() with a duration, so they stop by themselves and nothing stays on.

5.10 How to activate the photoresistor in Tinkercad

- Click **Start Simulation**.
- Click the photoresistor. A light slider appears.
- Start with the slider low (dark), then drag it up. When the reading passes 700, you hear the crossing sound once.
- To hear it again, drag the slider back down, then up.
- Open the **Serial Monitor** (bottom of the code panel) to watch the light= value as you move the slider. If the sound does not trigger where you want, change lightThreshold to a number between your dark and bright readings.
- If it still misbehaves, check that the photoresistor, the 10 k ohm resistor and the wire to A0 all share the same breadboard row.

6. Alert Key (what the user feels and hears)

Signal	Meaning
Low long tone	Pothole or drop ahead
Silence	Nothing detected
Beeps, faster when closer	Obstacle ahead
Vibration together with the beeps	The obstacle is likely living
Two rising tones (once)	Crossing

7. Testing in Tinkercad

- Build the circuit, click **Code**, switch to **Text**, paste the code, then click **Start Simulation**.
- Click each ultrasonic sensor and drag the object to change the distance.
- Start with the ground sensor at its normal floor distance. Then move it more than 20 cm farther to get the low tone (pothole).
- Front distance under 100 cm gives obstacle beeps, faster as you get closer.
- Front distance under 100 cm and a triggered PIR gives the beeps plus motor vibration (living).
- Start with the LDR light slider low, then raise it above about 700 for the crossing melody (once). Lower and raise it again to repeat.
- Use the Serial Monitor to see the front, ground and light values.

8. Things You Can Tune

- **groundBase**: measured automatically at start-up. Begin the simulation with the ground sensor at its normal floor distance.
- **+ 20**: the pothole tolerance. Smaller means more sensitive.
- **100** in front < 100: how early the obstacle warning begins.
- **lightThreshold (700)**: the light level for detecting a crossing. Use the Serial Monitor to pick a value between dark and bright readings.

9. Limitations and Next Steps

- Tinkercad has no vibration motor, so a DC motor stands in. A simpler stand-in is a second LED on D5 with a 220 ohm resistor.
- The delay() calls pause the whole program. A more advanced version would use millis().
- The LDR can be fooled by sunlight or shiny pavement. A real device would need a proper line or stripe sensor.
- A PIR only detects movement of warm bodies, so a still person may be missed.
- A possible upgrade is adding left and right sensors for direction.